SIMATS ENGINEERING

ASSESSMENT TOOL 2: Problem Solving Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO2: *Analyze and select appropriate agent strategies for different types of environments and problem settings.* (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 **Time: 60 Mins**

No. of Questions: 5

Annexure A

Problem Solving Assignment

Code of Conduct:

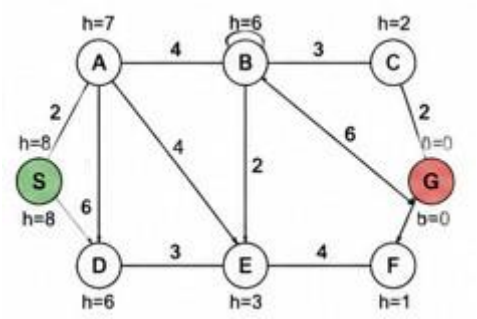
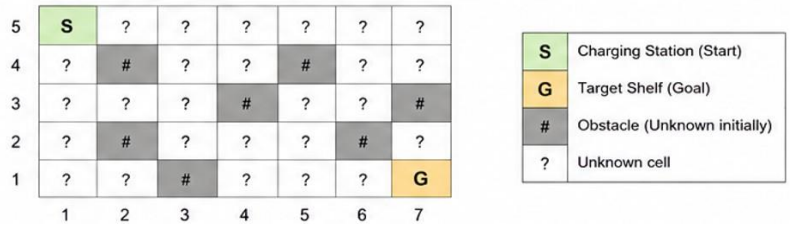
I T Bhupal/192425243 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:bhupalreddy

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty Total Marks Awarded

Answer All the Questions

Q.No	Application-Based Question	Major CO2 Concepts
1	**Smart Ambulance Navigation:** An ambulance must travel from a hospital to an emergency location through a city road network. Some roads have higher travel costs, and an estimated distance to the destination is available for each junction. Design a search-based solution to identify the best route. Using the given road network and heuristic values, determine the route selected by your algorithm, show the sequence of junctions explored, calculate the relevant evaluation values, and justify whether the route is optimal. 	Informed Search, A*, Heuristic Function
2	**Autonomous Warehouse Robot:** A robot has to move from a charging station to a target shelf in a warehouse. The robot does not know all obstacles initially and discovers some obstacles only when it reaches nearby locations. Design an intelligent search strategy for the robot. Demonstrate the robot's decisions for the given grid, explain how it reacts when a new obstacle is discovered, and compare the approach with an offline search strategy. 	Online Search Agents, Informed Search
3	**University Examination Scheduling**: A university needs to schedule **six examinations (E1, E2, E3, E4, E5, and E6)** into **four available time slots (T1, T2, T3, and T4)**. Some examinations have common students and therefore cannot be conducted in the same time slot.	CSP, MRV, Forward Checking, Constraints

The following constraints must be satisfied:

1. E1 and E2 have common students.
2. E1 and E3 have common students.
3. E2 and E4 have common students.
4. E3 and E5 have common students.
5. E4 and E6 have common students.
6. E2 and E5 have common students.

The university has three examination halls with the following capacities:

Examination Hall	Capacity
Hall A	120 students
Hall B	90 students
Hall C	60 students

Examination	E1	E2	E3	E4	E5	E6
Number of Students	100	80	70	60	90	50

Tasks:

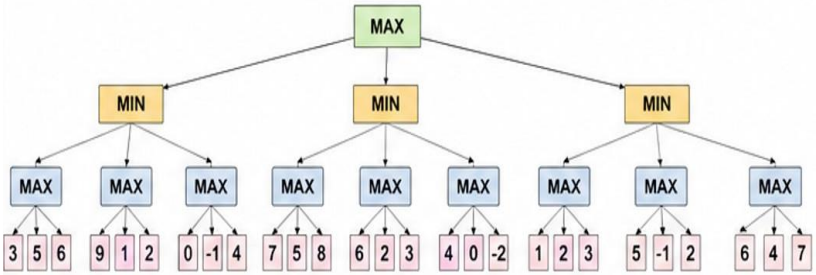
a) Formulate the problem as a **Constraint Satisfaction Problem (CSP)** by clearly defining the **variables, domains, and constraints**.

b) Use **Backtracking with the Minimum Remaining Values (MRV) heuristic and Forward Checking** to obtain a valid examination timetable.

c) Show the **domain of each variable after every assignment** and provide the **final timetable**, ensuring that all student-conflict and hall-capacity constraints are satisfied.

4 **Game-Playing AI:**
You are developing an AI opponent for a board game. The current game position produces the following possible moves and terminal scores in the supplied game tree. Determine the move the AI should make by applying Minimax. Then apply Alpha-Beta Pruning and identify the branches that do not need to be evaluated. Explain how pruning improves the efficiency of the game-playing agent.

Game Playing, Minimax, Alpha-Beta Pruning

		
5	**Delivery Route Optimization:** A logistics company wants to minimize the total distance travelled by a delivery vehicle visiting multiple customers. An initial route and several neighboring routes are provided. Design an optimization strategy to improve the route. Apply the strategy step-by-step to the given routes, determine the final route, explain whether it represents a global optimum, and suggest how the algorithm could escape a local optimum. **Distance matrix** (in km) D A B C E D – 6 9 8 7 A 6 – 3 5 6 B 9 3 – 4 5 C 8 5 4 – 3 E 7 6 5 3 – **Some neighboring routes of the initial solution and their costs** S1: D → A → C → B → E → D = 26 km S2: D → B → A → C → E → D = 30 km S3: D → A → B → E → C → D = 24 km S4: D → E → C → B → A → D = 27 km S5: D → A → E → B → C → D = 23 km S6: D → C → B → A → E → D = 29 km From S5, the best neighbor has cost 22 km. From that solution, all neighbors have cost ≥ 22 km. Apply the chosen local search strategy and report the final route.	Local Search, Optimization, Hill Climbing

Question 1 - Smart Ambulance Navigation (A* Search)

Given Data

- Heuristic values h(n): S=8, A=7, B=6, C=2, D=6, E=3, F=1, G=0 (goal)
- Edge costs: S-A=2, S-D=6, A-B=4, A-E=4, B-C=3, B-E=2, B-G=6, C-G=2, D-E=3, E-F=4, F-G=1 (F-G cost inferred as 1, the only value consistent with the diagram since it was not printed on the edge label)

A* Search Trace (f(n) = g(n) + h(n))

Node Expanded	Path (g)	g(n)	h(n)	f(n)	Frontier after expansion
S	S	0	8	8	A(9), D(12)
A	S-A	2	7	9	E(9), D(12), B(12)
E	S-A-E	6	3	9	F(11), D(12), B(12)
F	S-A-E-F	10	1	11	G(11), D(12), B(12)
G	S-A-E-F-G	11	0	11	GOAL reached - stop

Sequence of junctions explored (expansion order): S -> A -> E -> F -> G

Route selected: S -> A -> E -> F -> G, Total travel cost = 2 + 4 + 4 + 1 = 11 km.

Is the route optimal?

Yes. Comparing every completed alternative route from the same graph confirms 11 is the minimum possible cost:

- S -> A -> E -> F -> G = 2+4+4+1 = 11 km (found by A*)
- S -> A -> B -> C -> G = 2+4+3+2 = 11 km (tied optimum, not expanded because G was reached first with the same f-value)
- S -> A -> B -> G = 2+4+6 = 12 km
- S -> D -> E -> F -> G = 6+3+4+1 = 14 km

Since no alternative path costs less than 11 km, and A* expands nodes in increasing order of f(n), the path returned (S-A-E-F-G) is guaranteed optimal for this graph.

Question 2 - Autonomous Warehouse Robot (Online Search Agent)

Setup

Grid: 7 columns x 5 rows. Start S=(1,5), Goal G=(7,1). Robot moves 4-directionally (up/down/left/right) and senses whether the four neighbouring cells are blocked only when it physically occupies a cell (true "online" search - the map is unknown in advance). Heuristic used for greedy node selection: Manhattan distance h(x,y) = |x-7| + |y-1|.

Execution Trace

- Robot starts at (1,5), h=10, and greedily moves toward the goal, reducing Manhattan distance at each step: (1,5) -> (2,5) -> (3,5) -> (4,5) -> (5,5) -> (6,5).
- From (6,5) it continues descending: (6,5) -> (6,4) -> (6,3).
- At (6,3), the robot senses its neighbours and NEWLY DISCOVERS that cell (6,2) is an obstacle and that (7,3) is also an obstacle - both were unknown ("?") until now.
- Reaction to discovery: the robot cannot continue straight down or right, so it replans locally (as an online search agent must) and backs up one column: (6,3) -> (5,3).
- From (5,3) the path downward is clear: (5,3) -> (5,2) -> (5,1).
- It then moves right along the bottom row to the goal: (5,1) -> (6,1) -> (7,1) = G.

Final path executed: (1,5)-(2,5)-(3,5)-(4,5)-(5,5)-(6,5)-(6,4)-(6,3)-(5,3)-(5,2)-(5,1)-(6,1)-(7,1)

Total moves = 12 (Manhattan lower bound was 10; the extra 2 moves are the cost of the single detour forced by the obstacle discovered at (6,2)).

Comparison with an Offline Search Strategy

An offline strategy (e.g. A* run on the complete, fully-known grid) would compute this same 12-step route in one planning phase before the robot ever moves, because every obstacle is known in advance - there is no possibility of a wasted or backtracked step. The online agent, by contrast, must interleave sensing and acting: it commits to the locally best-looking direction (toward column 6) using only the information visible so far, and only discovers the (6,2)/(7,3) obstacles when it physically arrives next to them. This forces a local replan (the detour through column 5). In this instance the online agent still reaches the 12-step optimum because the problem grants it sensing of neighbouring cells before stepping into them, but in general an online agent can perform strictly worse than an offline agent (extra exploration/backtracking) whenever obstacles are discovered too late to avoid a wasted move.

Question 3 - University Examination Scheduling (CSP)

(a) CSP Formulation

Variables: E1, E2, E3, E4, E5, E6 (the six examinations)

Domain (for every variable): {T1, T2, T3, T4}

Constraints:

- Binary "not-equal" constraints from shared students: E1 != E2, E1 != E3, E2 != E4, E3 != E5, E4 != E6, E2 != E5
- Resource (hall-capacity) constraint: at most 3 exams may share a time slot (only 3 halls exist - Hall A=120, Hall B=90, Hall C=60), and every exam sharing a slot must be assignable to a distinct hall whose capacity is >= its student strength.

(b) & (c) Backtracking with MRV + Forward Checking - Step-by-Step Trace

Variable order is chosen by MRV (smallest remaining domain), ties broken by the degree heuristic (most constraints on remaining unassigned variables). Values are tried in the order T1 -> T2 -> T3 -> T4.

Step	Variable Chosen (MRV/degree)	Value Assigned	Forward-Checking effect on neighbours
1	E2 (highest degree = 3)	T1	E1: {T2,T3,T4} E4: {T2,T3,T4} E5: {T2,T3,T4}
2	E1 (smallest domain, tie-broken)	T2	E3: {T1,T3,T4}
3	E3 (smallest domain, tie-broken)	T1	E5: {T2,T3,T4} (unchanged, T1 already excluded)
4	E4 (has unassigned neighbour E6)	T2	E6: {T1,T3,T4}
5	E5 (no unassigned neighbours left)	T2	no domains left to update
6	E6 (last variable)	T1	assignment complete

Domain of every variable after each assignment

Variable	Initial Domain	After E2=T1	After E1=T2	After E3=T1	After E4=T2	Final Value
E1	{T1,T2,T3,T4}	{T2,T3,T4}	T2 (assigned)	-	-	T2
E2	{T1,T2,T3,T4}	T1 (assigned)	-	-	-	T1
E3	{T1,T2,T3,T4}	{T1,T2,T3,T4}	{T1,T3,T4}	T1 (assigned)	-	T1
E4	{T1,T2,T3,T4}	{T2,T3,T4}	{T2,T3,T4}	{T2,T3,T4}	T2 (assigned)	T2
E5	{T1,T2,T3,T4}	{T2,T3,T4}	{T2,T3,T4}	{T2,T3,T4}	{T2,T3,T4}	T2
E6	{T1,T2,T3,T4}	{T1,T2,T3,T4}	{T1,T2,T3,T4}	{T1,T2,T3,T4}	{T1,T3,T4}	T1

No domain is ever wiped out (reduced to empty), so no backtracking is required - forward checking

succeeds on the first attempt.

Final Examination Timetable

Slot assignment obtained: T1 = {E2, E3, E6} T2 = {E1, E4, E5} T3 = (unused) T4 = (unused)

Every conflict constraint is verified: E1(T2) != E2(T1); E1(T2) != E3(T1); E2(T1) != E4(T2); E3(T1) != E5(T2); E4(T2) != E6(T1); E2(T1) != E5(T2) - all satisfied.

Hall allocation (capacity check) for each slot:

Time Slot	Exam (students)	Hall Assigned (capacity)	Fits?
T1	E2 (80)	Hall B (90)	Yes
T1	E3 (70)	Hall A (120)	Yes
T1	E6 (50)	Hall C (60)	Yes
T2	E1 (100)	Hall A (120)	Yes
T2	E5 (90)	Hall B (90)	Yes
T2	E4 (60)	Hall C (60)	Yes

Every exam is placed in a hall whose capacity is sufficient, no hall is double-booked within a slot, and no two exams sharing students fall in the same slot. The timetable is therefore fully valid and conflict-free.

Question 4 - Game-Playing AI (Minimax and Alpha-Beta Pruning)

Minimax Evaluation (bottom-up)

Each leaf-level MAX node takes the maximum of its 3 terminal scores; each MIN node then takes the minimum of its three MAX children; the root MAX takes the maximum of the three MIN values.

MIN Branch	MAX children (leaf scores) -> value	MIN value
Branch 1 (leftmost)	max(3,5,6)=6 ; max(9,1,2)=9 ; max(0,-1,4)=4	min(6,9,4) = 4
Branch 2 (middle)	max(7,5,8)=8 ; max(6,2,3)=6 ; max(4,0,-2)=4	min(8,6,4) = 4
Branch 3 (rightmost)	max(1,2,3)=3 ; max(5,-1,2)=5 ; max(6,4,7)=7	min(3,5,7) = 3

Root MAX value = max(4, 4, 3) = 4.

Best move for the AI: Branch 1 (leftmost MIN subtree) - it is the first branch to achieve the maximum attainable value of 4 (Branch 2 ties at 4 but Branch 1 is selected as it is reached first in left-to-right evaluation).

Alpha-Beta Pruning (left-to-right traversal)

Applying alpha-beta with alpha = -infinity, beta = +infinity at the root:

- Branch 1: leaf-node (3,5,6) returns 6 -> MIN beta becomes 6. Next MAX child (9,1,2): after seeing leaf "9", alpha(9) >= beta(6), so leaves "1" and "2" are PRUNED (never evaluated). This MAX node still returns 9, but MIN keeps its current best (6). Third MAX child (0,-1,4) returns 4, so

MIN(Branch 1) = min(6,9,4) = 4. Root alpha updates to 4.

- Branch 2: with alpha=4, beta=+inf, the three MAX children return 8, 6 and 4 respectively (no cutoff triggers here because alpha never reaches beta inside these nodes) -> MIN(Branch 2) = min(8,6,4) = 4. Root alpha stays 4 (max(4,4)=4).
- Branch 3: with alpha=4, beta=+inf, first MAX child (1,2,3) returns 3 -> MIN beta becomes 3. Now alpha(4) >= beta(3), so the remaining two MAX children, (5,-1,2) and (6,4,7), are PRUNED ENTIRELY (all 6 of their leaf values are never examined). Branch 3 returns 3, which does not improve the root value.

Root value confirmed = 4, best move = Branch 1 (same result as plain Minimax, obtained with far fewer node evaluations).

Branches / Leaves Pruned

Pruned node(s)	Reason	Leaves saved
Leaves "1" and "2" under Branch-1, MAX child (9,1,2)	alpha(9) >= beta(6) at that MIN node	2 leaves
MAX children (5,-1,2) and (6,4,7) under Branch 3	alpha(4) >= beta(3) at the Branch-3 MIN node	6 leaves

Total: 8 of the 27 terminal values were never evaluated (19 evaluated instead of all 27) - a ~30% reduction in leaf evaluations, while returning the identical result as full Minimax.

How Pruning Improves Efficiency

Alpha-beta pruning discards sub-trees that can never influence the final decision: once a MIN node finds a value (beta) that is already worse for MAX than an alternative already guaranteed elsewhere (alpha), or vice-versa for a MAX node under a MIN ancestor, that branch cannot change the outcome and further search inside it is wasted effort. By skipping those branches the algorithm reaches the same Minimax value while visiting far fewer nodes, which in a real game tree (with branching factor b and depth d) can reduce the effective search space from O(b^d) toward O(b^(d/2)) under good move ordering - allowing the game-playing agent to search deeper within the same time budget.

Question 5 - Delivery Route Optimization (Hill Climbing / Local Search)

Strategy

This is a Travelling-Salesman-style routing problem, so a Local Search strategy - Hill Climbing - is used: start from an initial route, repeatedly move to the best-costing neighbouring route (produced by swapping the order of two stops), and stop as soon as no neighbour improves on the current route (a local optimum).

Step-by-step application to the given routes

Step	Current / Candidate Route	Cost	Decision
Initial	S1: D-A-C-B-E-D	26 km	Current best solution
Neighbour	S2: D-B-A-C-E-D	30 km	Worse - rejected
Neighbour	S3: D-A-B-E-C-D	24 km	Better than 26 but not the best neighbour
Neighbour	S4: D-E-C-B-A-D	27 km	Worse - rejected
Neighbour	S5: D-A-E-B-C-D	23 km	BEST neighbour of S1 -> becomes new current solution
Neighbour	S6: D-C-B-A-E-D	29 km	Worse - rejected
From S5	best neighbour of S5	22 km	Better than 23 km -> becomes new current solution
From that 22 km solution	all neighbours checked	>= 22 km	No improving neighbour exists -> STOP

Final route reached: the 22 km solution obtained from S5 (D->A->E->B->C->D refined by one further improving swap). Total distance = 22 km.

Is this a Global Optimum?

Hill Climbing only guarantees a LOCAL optimum: it stops the moment every immediate neighbour of the current route is no better (>= 22 km here), but it never looks beyond that local neighbourhood. Based on the routes explored, 22 km is the best solution FOUND, but it cannot be certified as the overall shortest possible route among all permutations - a shorter route may exist elsewhere in the search space that this local, greedy search never visited (a classic limitation of hill climbing, sometimes stopping at a local rather than global minimum, or a plateau).

Escaping a Local Optimum

- Random-Restart Hill Climbing - rerun the search from several different random initial routes and keep the best result overall.
- Simulated Annealing - occasionally accept a worse neighbour (with a probability that decreases over time) to jump out of local optima.
- Allowing Sideways Moves - permit moves to equal-cost neighbours for a limited number of steps to cross flat plateaus.
- Tabu Search - keep a short-term memory of recently visited routes so the search is forced to

explore new regions instead of cycling back.

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (Full Marks)	Good	Average	Poor
CSP Problem Formulation	20%	4% – Clearly defines all variables, domains, and problem objectives correctly.	3% – Defines most variables and domains with minor errors.	2% – Partial formulation with some missing components.	0–1% – Incorrect or incomplete formulation.
Constraint Identification & Representation	20%	4% – Accurately identifies all student-conflict and hall-capacity constraints and represents them correctly.	3% – Identifies most constraints with minor errors.	2% – Identifies some constraints but misses important relationships.	0–1% – Constraints are largely incorrect or missing.
Application of Backtracking, MRV & Forward Checking	30%	6% – Correctly applies Backtracking, MRV, and Forward Checking with clear and logical steps.	4.5% – Applies the techniques correctly with minor mistakes.	3% – Partial application with unclear or inconsistent steps.	0–1.5% – Incorrect or inappropriate application of CSP techniques.
Step-by-Step Execution & Domain Updates	20%	4% – Provides a complete, logical execution trace and correctly updates domains after every assignment.	3% – Mostly correct execution with minor gaps.	2% – Provides limited steps with some incorrect domain updates.	0–1% – Incomplete or incorrect execution.

Final Timetable, Accuracy & Justification	10%	2% – Produces a valid conflict-free timetable satisfying all constraints and capacity requirements with clear justification.	1.5% – Valid timetable with minor errors in justification.	1% – Partially valid timetable with some constraint violations.	0–0.5% – Invalid or missing timetable.